SECTION 1: Workflow Trigger Process

1.1 Supported triggers (actual)

The workflow runs on:

1) **Push** to branches:

- `main`
- `master`
- `feature/ci-pipeline`

2) **Pull Request** targeting:

- `main`
- `master`

3) **Manual run** via `workflow_dispatch`

1.2 Manual inputs (workflow_dispatch)

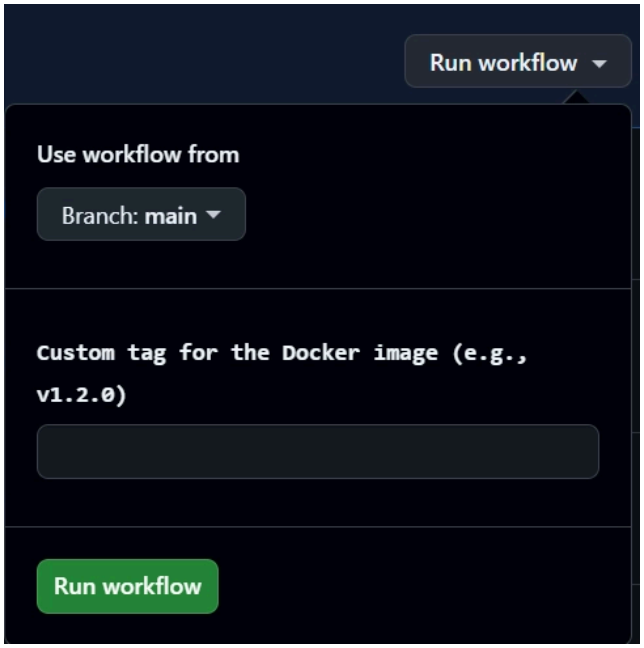
Input	Type	Allowed	Purpose
`service`	choice/string	one microservice name OR `all`	Release one service independently or all services in parallel
`image_tag`	string	optional (e.g., `v1.2.0`)	Overrides auto-generated tag (highest priority)

Operational note: DevOps can release a single microservice without affecting other services because the pipeline prepares a per-service matrix and builds/scans/pushes each service as an isolated artifact.

1.3 Sample YAML snippet (trigger + inputs)

```yaml
on:
  push:
    branches:
```

```yaml
      - main
      - master
      - feature/ci-pipeline
  pull_request:
    branches:
      - main
      - master
  workflow_dispatch:
    inputs:
      service:
        description: "Microservice to build (or all)"
        required: true
        default: "all"
      image_tag:
        description: "Optional custom image tag (e.g., v1.2.
0)"
        required: false
```



SECTION 2: Service Matrix Preparation

2.1 What the “prepare matrix” job does

The first job builds a **matrix** of microservices to process.

- If `service = all`: matrix contains **all microservices** → builds run **in parallel**
- If a single service is selected: matrix contains **only that service** → build/scans/push only for that service

2.2 Why matrix is used (release/operations)

- Reduces total release time via parallelism
- Matches microservice architecture (independent build + publish)
- Enables targeted hotfix releases (single service only)

2.3 Example matrix logic (GitHub Actions style)

```
jobs:
  prepare:
    runs-on: ubuntu-latest
    outputs:
      matrix: $ steps.set-matrix.outputs.matrix
    steps:
      - id: set-matrix
        run: |
          if [ "$ github.event.inputs.service " = "all" ]; th
en
            echo 'matrix=["api-gateway","user-service","authe
ntication-service","product-service","search-service","invent
ory-service","cart-service","wishlist-service","order-servic
e","payment-service","invoice-service","shipping-service","re
view-rating-service","recommendation-service","admin-servic
e"]' >> $GITHUB_OUTPUT
          else
            echo 'matrix=["$ github.event.inputs.service "]'
```

```
>> $GITHUB_OUTPUT
          fi

  build_scan_push:
    needs: prepare
    strategy:
      fail-fast: false
      matrix:
        service: $ fromJson(needs.prepare.outputs.matrix)
```

SECTION 3: Dockerfile Standardization

3.1 Dockerfile per microservice (actual)

Each microservice has its **own Dockerfile**.

Location standard:

`ecommerce-microservices/<service>/Dockerfile`

Examples:

- `ecommerce-microservices/product-service/Dockerfile`
- `ecommerce-microservices/cart-service/Dockerfile`
- `ecommerce-microservices/payment-service/Dockerfile`

3.2 Sample production Dockerfile (Node.js service)

```
FROM node:20-alpine
WORKDIR /app

COPY package*.json ./
RUN npm install --production

COPY . .
EXPOSE 3000
CMD ["npm","start"]
```

3.3 Image standards enforced (release expectations)

- Prefer **lightweight base images** (e.g., `alpine`)
- Keep build context minimal (use `.dockerignore`)
- Avoid embedding secrets into layers
- Version tag is immutable; `latest` is the moving pointer

SECTION 4: Dynamic Image Tagging Strategy

4.1 Tag priority (actual logic)

The pipeline generates the final image tag using this priority order:

Priority 1: Manual `image_tag` input (highest priority)

- Example: `v1.2.0`

Priority 2: Git release tag (if workflow run is on a git tag)

- Example ref: `refs/tags/v2.0.1`
- Resolved tag value: `v2.0.1`

Priority 3: Git short commit hash (fallback)

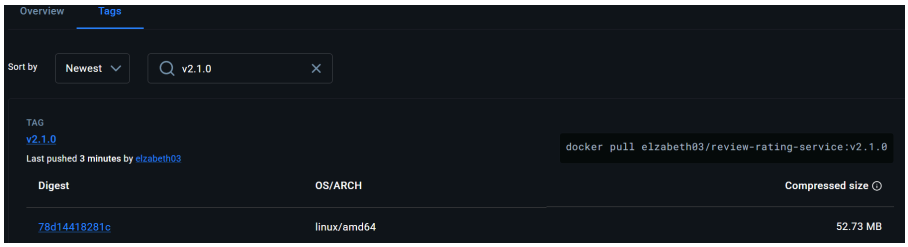
- Example: `3fa91bc`

4.2 Final image names (examples)

- `mydockerhub/product-service:v1.2.0`
- `mydockerhub/product-service:v2.0.1`
- `mydockerhub/product-service:3fa91bc`
- `mydockerhub/product-service:latest`

4.3 Why this enables rollbacks (operational value)

- **Version tag** is immutable → any prior release can be redeployed
- `latest` is only the “current stable” marker, not rollback-safe by itself



SECTION 5: Docker Build Process

5.1 Build tool used in CI

- Docker Build is executed in GitHub Actions runners
- Build is performed per selected microservice via matrix
- Strategy is configured with `fail-fast: false` so one failing service does not block other services from completing (parallel resilience)

5.2 Actual build pattern (command form)

```
docker build \
  -t DOCKER_USERNAME/product-service:<tag> \
  -f ecommerce-microservices/product-service/Dockerfile \
  ecommerce-microservices/product-service
```

5.3 Operational notes

- Build context is **service folder** (keeps builds isolated)
- Tag naming is consistent across services: `DOCKER_USERNAME/<service>:<tag>`

SECTION 6: Image Artifact Handling

6.1 Why we export the image as an artifact

After build, the pipeline **saves the Docker image** into a tar file and uploads it as a workflow artifact.

Purpose:

1. Reuse the exact same built image in later jobs
2. Avoid rebuilding (time + consistency)
3. Ensure scan and push operate on identical binary image
4. Faster end-to-end CI pipeline

6.2 Actual commands used

Save image:

```
docker save IMAGE_NAME > image.tar
```

Later (another job):

```
docker load < image.tar
```

6.3 Artifact actions used

- Upload: `actions/upload-artifact`
- Download: `actions/download-artifact`

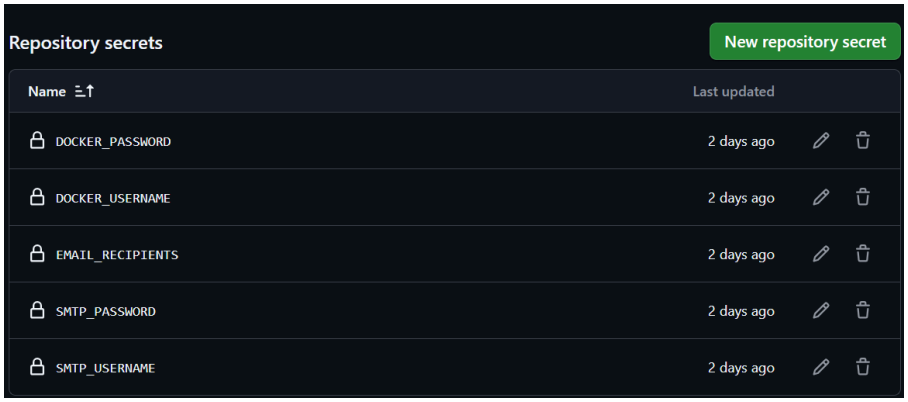
SECTION 7: GitHub Secrets Management

7.1 Secrets used (real)

Secret	Used for	Where used
`DOCKER_USERNAME`	DockerHub username / repo namespace	tagging + docker login
`DOCKER_PASSWORD`	DockerHub authentication token/password	`docker/login-action@v3`
`SMTP_USERNAME`	SMTP sender account username	email notification steps
`SMTP_PASSWORD`	SMTP app password / credential	email notification steps
`EMAIL_RECIPIENTS`	distribution list for alerts	failure + success notifications

7.2 Operational policy

- Secrets are stored in **GitHub repository encrypted secrets** (both organization-level and repository-level).
- Never hardcode credentials in workflow YAML.
- Never echo secrets to logs (avoid `set -x`, avoid printing env).



SECTION 8: Trivy Image Security Scan

8.1 When scan runs

Trivy scan runs **after Docker build** and **before DockerHub push**.

This ensures **no insecure image reaches DockerHub**.

8.2 Scan configuration (actual)

- Severity: `CRITICAL,HIGH`
- Scan types: `os,library`
- Ignore unfixed: `true`
- Exit code: `1` (pipeline fails if matching vulnerabilities found)

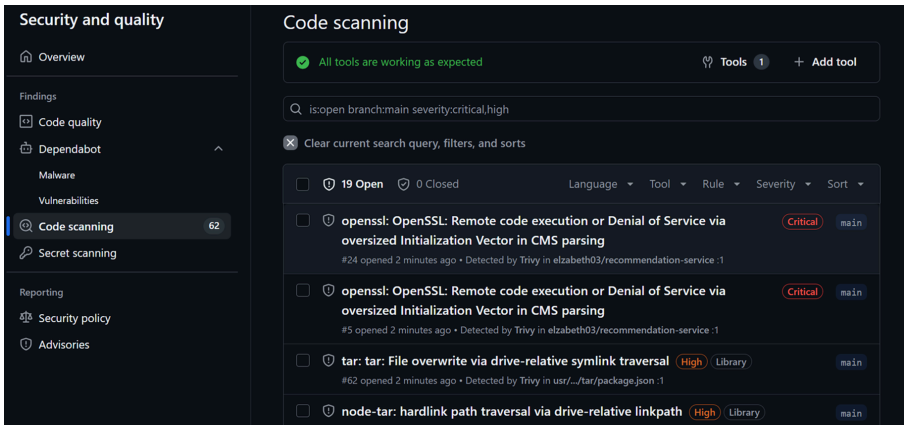
8.3 Sample Trivy command (local reproduction)

```
trivy image mydockerhub/product-service:v1.2.0
```

8.4 Release gate meaning

If any `HIGH` or `CRITICAL` vulnerability is detected:

- The pipeline job fails immediately for that service
- A failure email alert is triggered (see Section 9)



SECTION 9: Failure Alert Notification

9.1 Failure scenario (Trivy gate)

If the Trivy scan fails (HIGH/CRITICAL found):

- Pipeline status for that microservice becomes **Failed**
- SMTP email is sent to `EMAIL_RECIPIENTS`

9.2 Example email metadata (actual style)

Subject:

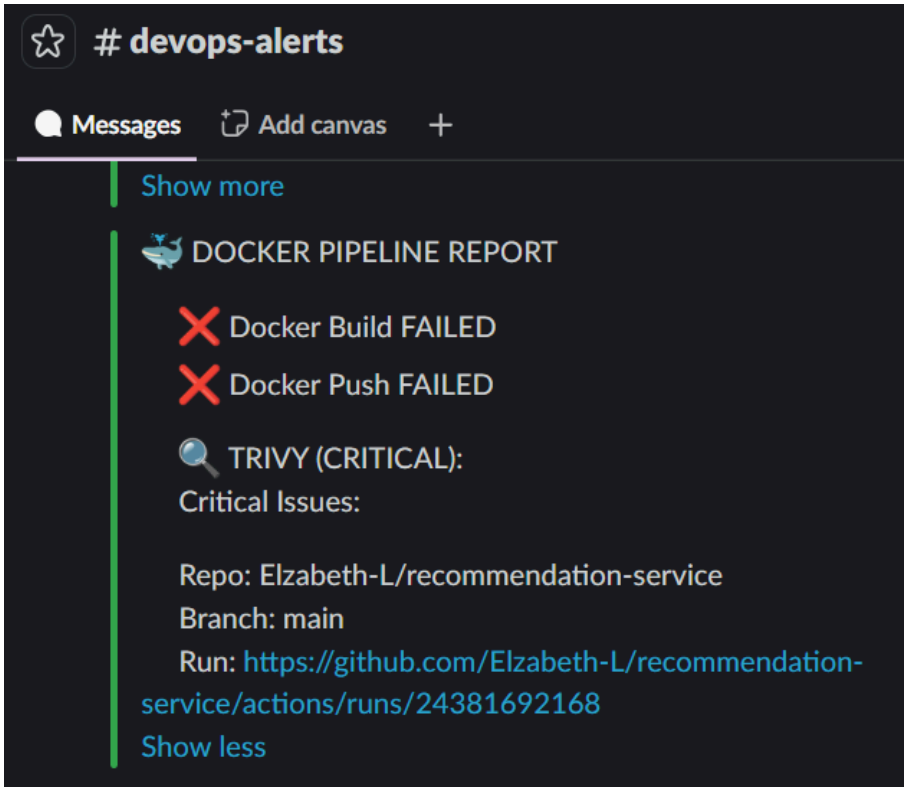
`CI Pipeline Failed (Trivy) - product-service`

Body (example content):

- Service: `product-service`
- Image tag: `<resolved tag>`
- Result: `HIGH/CRITICAL vulnerabilities found`
- Action: `Release blocked. Fix vulnerabilities and re-run.`

9.3 Why this reduces production risk

- Alerts reach DevOps immediately
- Prevents vulnerable artifacts from being published
- Enforces security as a release requirement



SECTION 10: DockerHub Authentication

10.1 When login occurs

DockerHub login happens **only after** the image passes Trivy scanning.

10.2 Authentication mechanism (actual)

Uses GitHub Action:

- `docker/login-action@v3`

Secrets:

- `username = DOCKER_USERNAME`
- `password = DOCKER_PASSWORD`

10.3 Sample YAML snippet (login)

```yaml
- name: DockerHub Login
  uses: docker/login-action@v3
  with:
    username: $ secrets.DOCKER_USERNAME
    password: $ secrets.DOCKER_PASSWORD
```

Security note: No credentials are printed; authentication occurs via encrypted secrets.

SECTION 11: Docker Push Process

11.1 Tags pushed (actual)

Two tags are pushed for each service:

1. **Version tag** (resolved tag from Section 4)
2. **latest**

11.2 Actual push commands (command form)

```
docker push mydockerhub/product-service:v1.2.0
docker push mydockerhub/product-service:latest
```

11.3 Operational meaning of each tag

Tag	Meaning	Usage
version tag	immutable release artifact	rollback, audit, promotion
latest	current stable pointer	default deploy reference (not rollback-safe)

SECTION 12: Final Success Notification

12.1 When success email is sent

After:

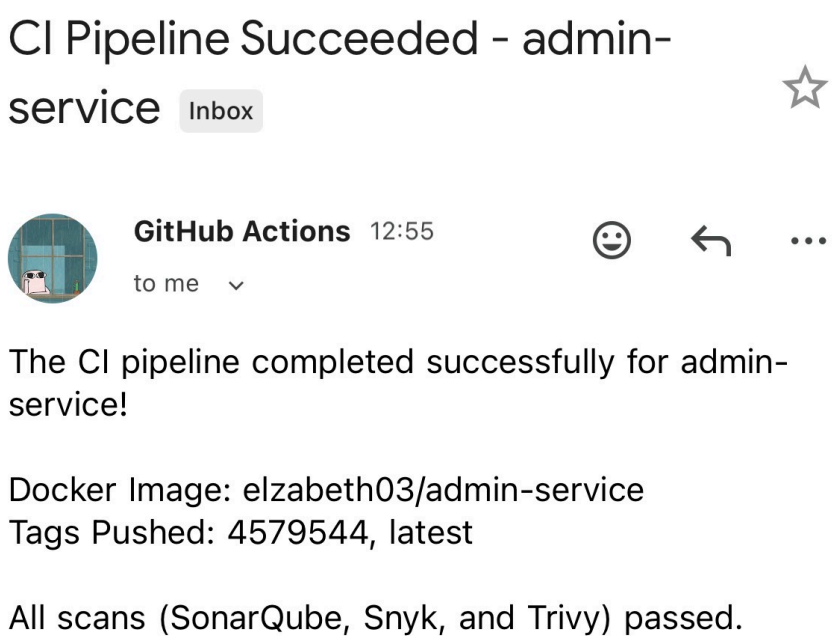
- Trivy scan passes, and
- DockerHub login succeeds, and
- Both version + latest tags are pushed successfully

12.2 Email content (what it includes)

- Service name (matrix service)
- Image name
- Version tag pushed
- `latest` pushed
- Pipeline status: Success

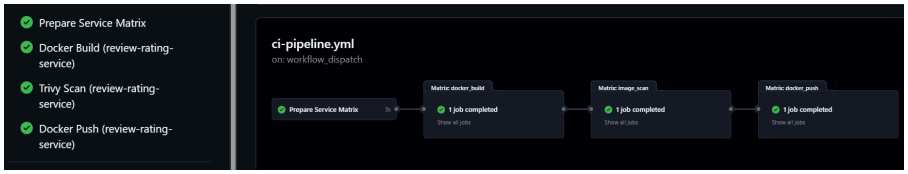
Example subject:

`CI Pipeline Succeeded - product-service`



SECTION 13: Security Best Practices Implemented

Control	Implementation in release workflow
Secrets protection	GitHub encrypted secrets (`DOCKER_*`, `SMTP_*`, `EMAIL_RECIPIENTS`)
No credentials in code	Credentials never stored in repo/YAML plaintext
Mandatory image scan	Trivy runs before push
Fail on severe issues	`exit-code: 1` for HIGH/CRITICAL
Immutable release	version tags based on input/tag/commit
Stable pointer	`latest` tag for current stable
Service isolation	separate Dockerfile + separate image per service
Artifact integrity	`docker save` → artifact → `docker load` for scan/push consistency
Minimal base images	lightweight images preferred (e.g., alpine)
Least privilege	credentials only used in relevant jobs/steps
Controlled registry access	push only after scan pass + authenticated login
Fast incident awareness	SMTP alerts on failure + success



SECTION 14: Common Issues & Fixes

14.1 Docker login failed

Symptom: `unauthorized: incorrect username or password`

Fix: Verify `DOCKER_USERNAME` and `DOCKER_PASSWORD` secrets; ensure token has correct permissions.

14.2 Push denied

Symptom: `denied: requested access to the resource is denied`

Fix: Confirm DockerHub repository exists and the user/token has push access to `DOCKER_USERNAME/<service>`.

14.3 Wrong tag generated

Symptom: Unexpected tag format in DockerHub

Fix: Validate `image_tag` input; confirm git tag naming conventions; confirm commit hash extraction.

14.4 Trivy failed (release blocked)

Symptom: Trivy finds HIGH/CRITICAL vulnerabilities

Fix (operational):

- Patch OS packages in Dockerfile/base image
- Update vulnerable dependencies
- Rebuild and re-run workflow after remediation

14.5 No email received

Symptom: Pipeline failed/succeeded but no SMTP email

Fix: Verify `SMTP_USERNAME`, `SMTP_PASSWORD`, `EMAIL_RECIPIENTS`; confirm Gmail SMTP/app-password settings; check spam/quarantine.

14.6 Artifact missing

Symptom: Download step cannot find artifact

Fix: Ensure upload/download artifact step names match exactly; ensure artifact naming includes service if required by matrix.

SECTION 15: End-to-End Release Flow

15.1 Operational release flow (actual)

Developer pushes code OR manual trigger →

Prepare service matrix →

(Upstream checks) SonarQube + Snyk →

Docker Build →

Save `image.tar` artifact →

Trivy Scan →

If fail: Email alert + stop release for that service →

If pass: DockerHub Login →

Push version tag + latest tag →

Success email →

ArgoCD later consumes the pushed image for Kubernetes deployment

---